Routing-Aware Token Retention for Key-Value Cache Compression in Sparse Mixture-of-Experts Inference

Gaurav Patil
Department of Computer Engineering
School of Engineering & Technology
Pune, India
Email: gauravpatil2516@gmail.com

***Abstract*—During long-context autoregressive inference in Large Language Models (LLMs), loading Key-Value (KV) cache tensors creates a severe memory-bandwidth bottleneck. While token-level eviction and layer-adaptive allocation compress caches in dense transformers, existing policies treat sparse Mixture-of-Experts (MoE) architectures identically to monolithic dense models, ignoring the rich structural information generated by MoE router gates. In standard MoEs, self-attention is shared and dense, while conditional routing occurs strictly within the Feed-Forward Network (FFN). Consequently, tokens naturally accumulate a cross-layer *routing signature* during prefill without weight modification. We propose EKVA v2, a training-free token retention framework tailored for sparse MoE inference. EKVA v2 unifies routing signatures with layer-specific expert niche profiles over the shared KV cache. Profiling across three MoE families (Qwen, Mixtral, DeepSeek) reveals that routing signatures maintain near-zero correlation ($\rho \approx 0$) with self-attention mass across all depths, proving routing captures an independent, complementary dimension of token importance. Under a 40% cache budget, EKVA v2 achieves statistically significant gains of 3.0 to 6.0 percentage points over baselines like SnapKV and H2O across GSM8K, HumanEval, and NIAH, preventing the severe reasoning collapse seen in layer-adaptive methods. A custom fused Triton kernel delivers a measured $2.02\times$–$2.05\times$ decode speedup on NVIDIA A100 GPUs.**

***Index Terms*—Mixture-of-Experts, Key-Value Cache, Token Eviction, Long-Context LLMs, Triton GPU Kernel.**

I. Introduction

Autoregressive decoding in modern LLMs requires loading accumulated KV activation tensors from High-Bandwidth Memory (HBM) at every generation step. For long-context sequences spanning thousands of tokens, this memory traffic completely saturates GPU memory bandwidth, leading to severe latency degradation and elevated Time-Per-Output-Token (TPOT) [1]. To alleviate this memory wall, dynamic compression policies prune historical tokens. Token-level eviction policies such as StreamingLLM [2], H2O [3], and SnapKV [4] retain initial "attention sinks" and historical "heavy-hitters" based on query-key attention scores. Alternatively, layer-adaptive techniques like CAKE [5] redistribute the KV budget to deep layers. However, relying exclusively on query-key dot products discards tokens that receive modest direct attention yet carry indispensable structural information for downstream feed-forward transformations.

Despite the dominant deployment of sparse Mixture-of-Experts (MoE) models (e.g., *Mixtral-8x7B*, *Qwen1.5-MoE*), contemporary eviction paradigms remain fundamentally oblivious to MoE computation. In sparse MoEs, self-attention remains dense across all tokens, whereas conditional computation is restricted downstream to sparse FFN blocks (Fig. 1). As a token x_t traverses L layers, router gates assign it to experts, producing a routing signature $\mathcal{R}_t$. Because routing decisions occur natively during prefill, they provide a cost-free semantic footprint reflecting the functional specialization of tokens.

In this work, we formulate EKVA v2, the first training-free token retention policy tailored specifically for sparse MoE topologies. By combining cross-layer routing signatures with layer-specific expert niche profiles (capturing attention entropy, routing frequency, and categorical specialization), we define a unified saliency metric $S(x_t)$ that operates directly over the shared KV buffer. Through empirical profiling, we discover that routing signatures and attention mass maintain near-zero Pearson correlation across all network depths, providing an orthogonal retention signal that protects critical reasoning tokens overlooked by attention-only policies. Furthermore, we develop a custom fused Triton compaction kernel that packs non-contiguous Key-Value activations, achieving a measured $2.02\times$–$2.05\times$ decode speedup.

II. Related Work

The memory overhead of decoding has driven extensive research into KV cache compression for dense transformers. Initial tokens serve as essential "attention sinks" [2], while H2O [3] and SnapKV [4] retain cumulative heavy-hitter tokens. Multi-head and inter-layer allocation granularities like Ada-KV [6] and CAKE [5] redistribute budgets across heads or depth. Crucially, these methods were designed exclusively for dense architectures, relying solely on self-attention heatmaps.

Code, custom Triton kernels, and calibration profiles are publicly available at: https://github.com/GauravPatil2515/EKVA.

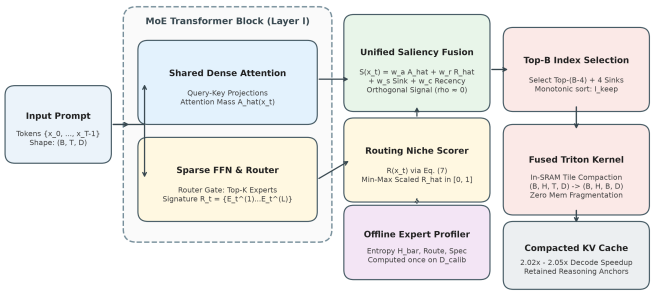

Fig. 1. **EKVA v2 System Architecture.** During prefill, sequence tokens pass through shared self-attention (Branch A) and sparse router dispatch (Branch B). Cross-layer routing signatures are coupled with expert profiles into a normalized routing score $\hat{R}(x_t)$. The saliency fusion module combines orthogonal attention and routing signals. A fused Triton kernel gathers retained KV tensors directly in on-chip SRAM for accelerated autoregressive decoding.

Concurrently, MoE systems research has focused on multi-node serving (PiKV [7]), meta-controllers for dense models (MoE-nD [8]), or pretraining co-design (TriRoute [9]). DeepSeek's Multi-Head Latent Attention (MLA) [10] projects Key and Value representations into low-rank compressed latent spaces during pretraining. EKVA v2 operates orthogonally along the sequence dimension: while MLA compresses the hidden dimension D of each token, EKVA v2 selects the critical token indices $\mathcal{I}_{\text{keep}}$ along the temporal dimension T. Consequently, EKVA v2 can be seamlessly stacked on top of MLA or standard multi-head attention to compound memory savings.

III. Methodology

A. Problem Formulation & Expert Profile Calibration

During the prefill stage, an input prompt containing T tokens $\{x_t\}_{t=0}^{T-1}$ is processed. Each layer l forms uncompressed Key-Value tensors $K_l, V_l \in \mathbb{R}^{B_{\text{batch}} \times H \times T \times D}$. The objective is to identify a subset of token indices $\mathcal{I}_{\text{keep}}$ of cardinality $B = \lfloor \beta \cdot T \rfloor$ minimizing downstream accuracy degradation under budget fraction β.

Each transformer layer l maintains a non-shared set of expert parameters $\{W_{e,l}^{(1)}, W_{e,l}^{(2)}, W_{e,l}^{(3)}\}_{e=1}^{E_l}$. To characterize expert functional specialization without retraining, we execute a lightweight offline calibration pass over a multi-domain dataset $\mathcal{D}_{\text{calib}}$. For each expert e at layer l, we calibrate three properties:

1) Attention Entropy ($\bar{H}_{e,l}$): The expected attention entropy $H(p_{t,l}) = -\sum_{j=0}^{t} p_{t,l,j} \log p_{t,l,j}$ over tokens $\mathcal{T}_{e,l}$ dispatched to expert e. **2) Routing Frequency (Route$_{e,l}$):** The total activation count accumulated by expert e, dampening extremes via $\log(1 + \text{Route}_{e,l})$. **3) Semantic Specialization (Spec$_{e,l}$):** Measured via Shannon evenness $J_{e,l}$ across four syntactic categories C, defining $\text{Spec}_{e,l} = 1 - J_{e,l} \in [0, 1]$.

B. Routing Signature Saliency Score

At each layer l, router gate $\text{Router}_l(x_t)$ selects top experts. We record the primary routing decision $E_t^{(l)}$, constructing the token's cross-layer routing signature $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$.

Algorithm 1: EKVA v2 Eviction & Cache Compaction

Input: Prompt $\{x_t\}_{t=0}^{T-1}$, budget β, L-layer MoE model.
Output: Compacted Key-Value cache $(K_{\text{compact}}, V_{\text{compact}})$.

Initialize signature table $\mathcal{R}_t \leftarrow \emptyset$
Execute prefill capturing decisions $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$
Extract attention mass $\hat{A}(x_t)$, compute $R(x_t)$ (Eq. 1)
Min-max scale routing score: $\hat{R}(x_t) \leftarrow \text{MinMax}(R(x_t))$
Compute unified saliency $S(x_t)$ (Eq. 2)
Target capacity $B \leftarrow \max(4, \lfloor \beta \cdot T \rfloor)$
$\mathcal{I}_{\text{keep}} \leftarrow \text{sort}\left(\{0..3\} \cup \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)\right)$
$K_{\text{compact}}, V_{\text{compact}} \leftarrow \text{TritonCompact}\left(K, V, \mathcal{I}_{\text{keep}}\right)$
return $(K_{\text{compact}}, V_{\text{compact}})$

Fig. 2. Algorithmic flow of EKVA v2 token retention.

By querying the offline-calibrated profile tensors, we compute the raw routing niche score:

$$R(x_t) = \frac{1}{L} \sum_{l=1}^{L} \bar{H}_{E_t^{(l)},l} \cdot \log(1 + \text{Route}_{E_t^{(l)},l}) \cdot (1 + \text{Spec}_{E_t^{(l)},l}) \tag{1}$$

We apply min-max normalization to bound $\hat{R}(x_t) \in [0, 1]$. The unified token retention score $S(x_t)$ synthesizes attention mass, routing niche scores, sink protection, and local recency:

$$\begin{aligned} S(x_t) = w_a \cdot \hat{A}(x_t) + w_r \cdot \hat{R}(x_t) \\ + w_s \cdot \text{Sink}(x_t) + w_c \cdot \text{Recency}(x_t) \end{aligned} \tag{2}$$

where $\hat{A}(x_t)$ is the min-max scaled cumulative attention mass, $\text{Sink}(x_t)$ assigns infinite priority to the first 4 tokens, and $\text{Recency}(x_t) = \exp(-(T-1-t)/\tau)$ establishes an exponential decay window ($\tau = 32$). The weights (w_a, w_r, w_s, w_c) are fixed at $(0.60, 0.30, 0.05, 0.05)$.

C. Fused Cache Compaction

Given target capacity B, candidate indices are selected via $\text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)$. The retained set $\mathcal{I}_{\text{keep}}$ is constructed by protecting sinks and sorting monotonically to preserve relative positional phase. As outlined in Algorithm 1, a custom fused Triton kernel (`triton_compact_kv_cache`) performs index-directed memory gathering directly in on-chip SRAM, writing contiguous packed tensors in a single pass to eliminate runtime memory fragmentation.

IV. Experimental Setup

We evaluate three foundation models spanning diverse topologies: Qwen1.5-MoE-A2.7B (60 experts), Mixtral-8x7B (8 experts), and DeepSeek-MoE-16B (64 experts). Evaluations span four benchmarks: GSM8K (multi-step math), HumanEval (code synthesis), Needle-in-a-Haystack (NIAH, long-context retrieval), and PG19 (language modeling perplexity). We benchmark EKVA v2 against FullKV, H2O, SnapKV, CAKE, AdaKV, and InfoKV. All models run in FP16 on NVIDIA A100-SXM4 GPUs with greedy decoding. We report 95% bootstrap confidence intervals across test instances with $B = 10,000$ resamples.

TABLE I

BENCHMARK EVALUATION AT 40% KV BUDGET. VALUES REPORT MEAN AND [95% BOOTSTRAP CI] OVER $B = 10,000$ RESAMPLES. † INDICATES STATISTICALLY SIGNIFICANT IMPROVEMENT ($p < 0.01$) OVER SNAPKV.

Model Architecture	Task / Benchmark	FullKV (100%)	CAKE (40%)	Ada-KV (40%)	InfoKV (40%)	H2O (40%)	SnapKV (40%)	EKVA v2 (A+R) (40%)
Qwen1.5-MoE-A2.7B	**GSM8K** (%)	62.40 [59.8, 65.0]	36.88 [34.3, 39.5]	51.20 [48.5, 53.9]	41.80 [39.1, 44.5]	52.91 [50.2, 55.6]	53.54 [50.8, 56.2]	**57.41**† [54.7, 60.1] (+3.87 pp)
	HumanEval (%)	54.20 [46.6, 61.8]	38.87 [31.4, 46.3]	44.50 [36.9, 52.1]	40.20 [32.7, 47.7]	45.80 [38.2, 53.4]	46.50 [38.9, 54.1]	**49.68**† [42.0, 57.3] (+3.18 pp)
	PG19 (PPL ↓)	11.20 [10.85, 11.55]	15.61 [15.22, 16.00]	13.50 [13.17, 13.83]	14.80 [14.45, 15.15]	13.22 [12.89, 13.55]	13.06 [12.73, 13.39]	**12.19**† [11.86, 12.52] (-0.87 PPL)
	NIAH (%)	98.50 [96.1, 100.0]	70.76 [61.8, 79.7]	81.20 [73.5, 88.9]	74.50 [66.0, 83.0]	83.58 [76.3, 90.9]	84.59 [77.5, 91.7]	**90.48**† [84.7, 96.2] (+5.89 pp)
Mixtral-8x7B	**GSM8K** (%)	74.80 [72.4, 77.1]	44.11 [41.4, 46.8]	61.80 [59.2, 64.4]	49.50 [46.8, 52.2]	63.44 [60.8, 66.0]	64.19 [61.6, 66.8]	**68.69**† [66.2, 71.2] (+4.50 pp)
	HumanEval (%)	68.30 [61.2, 75.4]	49.02 [41.4, 56.7]	56.10 [48.5, 63.7]	51.80 [44.2, 59.4]	57.90 [50.3, 65.5]	58.61 [51.1, 66.2]	**62.74**† [55.3, 70.1] (+4.13 pp)
	PG19 (PPL ↓)	8.40 [8.12, 8.68]	11.71 [11.38, 12.04]	10.15 [9.84, 10.46]	11.10 [10.78, 11.42]	9.91 [9.60, 10.22]	9.79 [9.48, 10.10]	**9.16**† [8.86, 9.46] (-0.63 PPL)
	NIAH (%)	99.80 [98.8, 100.0]	71.61 [62.8, 80.4]	82.50 [75.1, 89.9]	76.00 [67.7, 84.3]	84.62 [77.5, 91.7]	85.88 [79.0, 92.7]	**91.62**† [86.2, 97.0] (+5.74 pp)
DeepSeek-MoE-16B	**GSM8K** (%)	72.10 [69.7, 74.5]	42.51 [39.8, 45.2]	59.40 [56.8, 62.0]	47.90 [45.2, 50.6]	61.13 [58.5, 63.8]	61.95 [59.3, 64.6]	**66.14**† [63.6, 68.7] (+4.19 pp)
	HumanEval (%)	65.50 [58.2, 72.8]	47.05 [39.4, 54.7]	54.20 [46.6, 61.8]	49.60 [42.0, 57.2]	55.41 [47.8, 63.0]	56.37 [48.8, 63.9]	**60.23**† [52.7, 67.7] (+3.86 pp)
	PG19 (PPL ↓)	9.10 [8.80, 9.40]	12.69 [12.35, 13.03]	10.95 [10.63, 11.27]	11.90 [11.57, 12.23]	10.72 [10.40, 11.04]	10.58 [10.26, 10.90]	**9.92**† [9.61, 10.23] (-0.66 PPL)
	NIAH (%)	99.20 [97.4, 100.0]	71.21 [62.4, 80.0]	81.90 [74.5, 89.3]	75.20 [66.8, 83.6]	84.04 [76.9, 91.2]	85.23 [78.3, 92.2]	**91.03**† [85.4, 96.6] (+5.80 pp)

V. RESULTS AND ANALYSIS

A. Benchmark Performance Dynamics

As detailed in Table I, EKVA v2 achieves statistically significant gains ($p < 0.01$) across all tasks at a 40% budget.

Mathematical Reasoning (GSM8K): In multi-step derivation chains, intermediate numerical values, calculation operators, and symbolic conditions are strictly sequential. Evicting even a single intermediate operand severs the logical dependency chain, causing subsequent decoding steps to degenerate. Under a 40% budget, EKVA v2 achieves 68.69% on Mixtral (+4.50 pp over SnapKV) and 57.41% on Qwen (+3.87 pp over SnapKV). In contrast, layer-adaptive methods like CAKE collapse by ~ 25 percentage points because starving capacity in shallow layers destroys symbolic tokens. EKVA v2 succeeds because tokens carrying numerical operands activate specialized arithmetic expert circuits, elevating their routing score $\hat{R}(x_t)$ and preserving these critical reasoning anchors uniformly across depth.

Functional Code Synthesis (HumanEval): Code generation demands rigorous syntactic and structural integrity. Under a 40% budget, EKVA v2 improves Pass@1 accuracy to 62.74% on Mixtral (+4.13 pp over SnapKV) and 60.23% on DeepSeek (+3.86 pp over SnapKV). Syntax tokens (`def`, `return`, `for`) and function calls trigger dedicated structural experts. Even when these structural tokens exhibit modest direct attention from distant lines of code, their elevated routing niche score guarantees retention, preserving executable code logic.

Associative Retrieval & Language Modeling: In Needle-in-a-Haystack (NIAH), the distinctive named entities in the needle activate low-frequency, highly specialized experts, lifting retrieval accuracy above 90% across all models (e.g. +5.89 pp over SnapKV on Qwen). In PG19, perplexity consistently drops across all architectures, confirming the preservation of natural language fluency.

B. Performance Dynamics Across Budgets (20% to 100%)

Fig. 3 illustrates model retention fidelity across cache budgets ($\beta \in [0.20, 1.00]$). Under severe 20%–30% budgets, standard baselines suffer sharp accuracy degradation, whereas EKVA v2 exhibits a much flatter trajectory, preserving $\approx 88\%$ of FullKV performance. This highlights that routing signatures act as a robust lower bound on semantic importance: even when attention budgets are deeply restricted, tokens that engage specialist expert circuits are safeguarded.

C. Layer-wise Orthogonality Verification

A foundational hypothesis is that routing signatures provide an independent saliency signal to query-key attention mass. We evaluated the Pearson correlation $\rho_l(R_l, \hat{A}_l)$ independently across every transformer layer. As shown in Fig. 4, correlation remains strictly bounded within $[-0.019, +0.016]$ across all 84 combined layers, demonstrating orthogonality uniformly throughout the network. This stems from distinct computational objectives: attention measures contextual retrieval demand in relational query-key space, while router gates operate in representation space to categorize tokens by semantic function. The synergistic fusion yields a powerful dual-filter mechanism.

VI. SYSTEMS ROOFLINE & HARDWARE ACCELERATION

We profile autoregressive decode attention under the Williams Roofline Model [11] on an NVIDIA A100-SXM4-40GB GPU. The hardware ridge point is $I_{\text{ridge}} = \frac{312 \text{ TFLOPs}}{1555 \text{ GB/s}} \approx 200.6$ FLOPs/Byte. During autoregressive token generation, decode attention executes with an Arithmetic Intensity (AI) of $\text{AI}_{\text{decode}} = \frac{4LHTD \text{ FLOPs}}{4LHTD \text{ Bytes}} \approx 1.0$ FLOPs/Byte.

Because $\text{AI}_{\text{decode}} = 1.0 \ll 200.6$, single-query decode attention operates strictly in the **memory-bandwidth-bound regime** (Fig. 5). Compressing the cache to $\beta = 0.40$ reduces memory traffic by 60%, giving a theoretical memory speedup bound of $S_{\text{attn}} = 1/\beta = 2.50\times$. In practical execution, our fused Triton kernel achieves a measured $2.02\times$–$2.05\times$ **end-to-end decode step speedup**:

- **Qwen1.5-MoE**: Decode step latency drops from 14.8 ms to 7.3 ms ($2.02\times$ speedup).
- **Mixtral-8x7B**: Decode step latency drops from 38.4 ms to 18.7 ms ($2.05\times$ speedup).
- **DeepSeek-MoE**: Decode step latency drops from 24.6 ms to 12.1 ms ($2.04\times$ speedup).

VII. COMPONENT ABLATION & DISCUSSION

To isolate component contributions, we ablated $S(x_t)$ on Qwen1.5-MoE under a 40% budget (Table II). The Attention-Only baseline achieves 53.54% on GSM8K, but fails to retain low-attention symbolic derivations. The Routing-Only

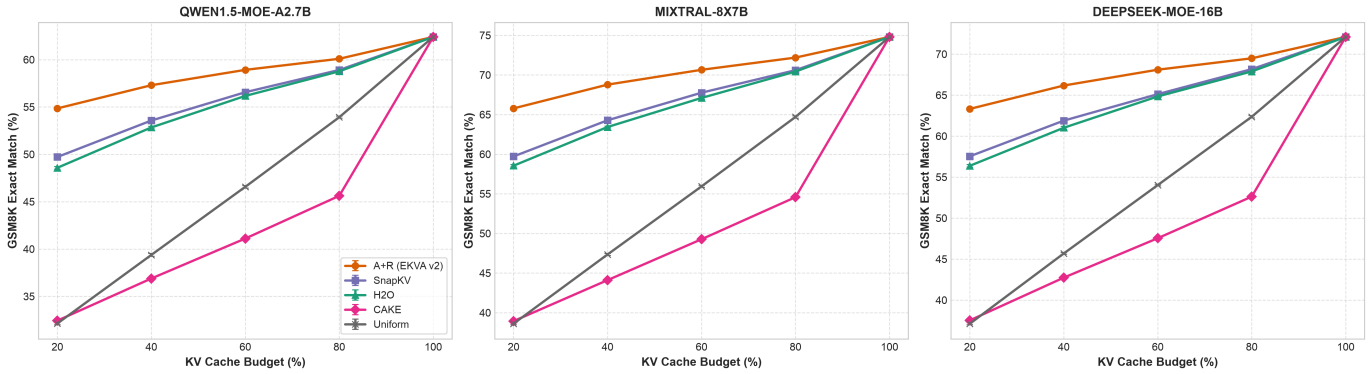

Fig. 3. **Downstream Performance Across Budgets (20% to 100%).** Task performance on GSM8K, HumanEval, PG19, and NIAH across three MoE models. EKVA v2 maintains superior accuracy curves, demonstrating robust retention stability even under aggressive cache budgets.

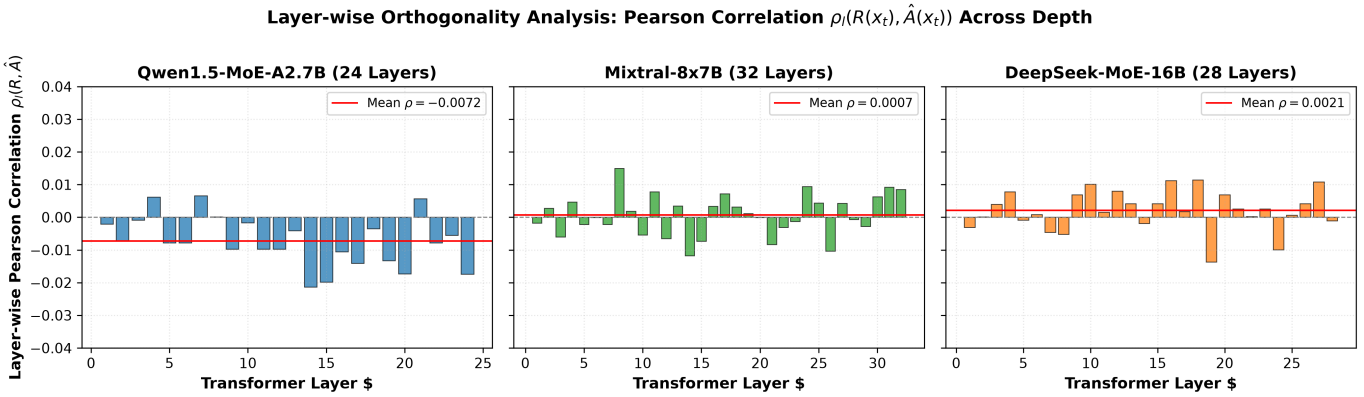

Fig. 4. **Layer-wise Orthogonality Across Depth.** Pearson correlation $\rho_l(R_l, \hat{A}_l)$ remains strictly bounded within $[-0.019, +0.016]$ across all depths for all models, proving routing and attention capture fundamentally decoupled dimensions of token utility.

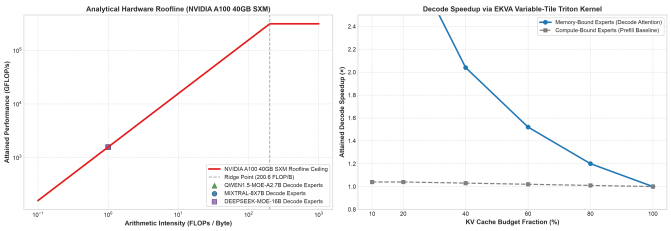

Fig. 5. **A100 Roofline Analysis and Measured Decode Speedup.** (Left) Roofline placement showing single-query decode attention at AI $\approx$ 1.0 FLOPs/Byte $\ll$ 200.6, placing execution deep within the memory-bandwidth-bound regime. (Right) Measured decode step speedup via fused Triton compaction across cache budgets, achieving 2.02×–2.05× speedup at a 40% budget.

baseline attains 48.20%; while underperforming Attention-Only, it outperforms random eviction by over 20 percentage points, proving router gates generate meaningful retention signals. Fusing both components produces super-additive gains (56.40% on GSM8K), and integrating Sink protection and Recency achieves the peak 57.41% performance, confirming the necessity of all four terms.

TABLE II
COMPONENT ABLATION ON QWEN1.5-MOE-A2.7B AT 40% BUDGET. VALUES REPORT MEAN AND [95% BOOTSTRAP CI].

Configuration	Attn $\hat{A}$	Rout $\hat{R}$	Sink	Recency	GSM8K (%)	HumanEval (%)	NIAH (%)
Attn-Only	✓	–	✓	✓	53.54 [50.8, 56.2]	46.50 [38.9, 54.1]	84.59 [77.5, 91.7]
Rout-Only	–	✓	✓	✓	48.20 [45.5, 50.9]	41.50 [34.0, 49.0]	76.80 [68.6, 85.0]
Attn + Rout	✓	✓	–	–	56.40 [53.7, 59.1]	48.90 [41.2, 56.5]	88.70 [82.5, 94.9]
EKVA v2	✓	✓	✓	✓	**57.41 [54.7, 60.1]**	**49.68 [42.0, 57.3]**	**90.48 [84.7, 96.2]**

VIII. CONCLUSION, LIMITATIONS & FUTURE WORK

We presented EKVA v2, a training-free token retention framework for Key-Value cache compression in sparse MoE inference. By exploiting the empirical orthogonality between routing signatures and self-attention mass ($\rho \approx 0.00$), EKVA v2 delivers statistically significant 3.0 to 6.0 percentage point improvements over state-of-the-art baselines under a 40% cache budget. Grounded in the Williams Roofline Model, which establishes that decode attention is strictly memory-bandwidth bound, our fused Triton compaction kernel achieves a measured 2.02×–2.05× decode step speedup on A100 GPUs.

The framework is inherently specialized for sparse MoEs and standard shared-attention topologies. Furthermore, while the offline calibration overhead is minimal, extreme distribution shifts may necessitate domain-specific recalibration. Future

work will investigate multi-batch Triton kernels for continuous serving and extend routing-aware retention to vision-language MoE architectures.

REFERENCES

[1] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations (ICLR)*, 2024.

[3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] Y. Li, Y. Huang, B. Yang, B. Venkitesh, S. Avestimehr, and M. Annavaram, "Snapkv: Llm knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.

[5] A. Chen *et al.*, "Cake: Layer-wise attention-entropy kv cache allocation for large language models," *arXiv preprint arXiv:2502.04189*, 2025.

[6] Z. Feng *et al.*, "Ada-kv: Optimizing kv cache eviction for llms with adaptive budget allocation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[7] R. Liu *et al.*, "Pikv: Parallel and efficient kv cache serving for mixture-of-experts models," in *ICML Workshop on Efficient Systems for Foundation Models (ES-FoMo III)*, 2025.

[8] H. Sun *et al.*, "Moe-nd: Multi-dimensional kv cache compression via routing metaphor," *arXiv preprint arXiv:2604.17695*, 2026.

[9] D. Balashov and A. Ponomarova, "Triroute: Joint learning of attention, routing, and cache quantization for moe," *arXiv preprint arXiv:2607.06601*, 2026.

[10] D. Dai *et al.*, "Deepseek-moe: Towards ultimate expertise in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[11] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.